

Pi LVGL Workflow Guide

คู่มือ Workflow สำหรับ Digital Calendar บน Raspberry Pi

Project:	Digital Calendar Pi
Hardware:	Raspberry Pi Zero WH (ARMv6, 512MB)
OS:	Raspberry Pi OS Lite Trixie 32-bit
Hostname:	digitalcal-pi
User:	thanat
Pi IP:	192.168.0.232 (LEnet WiFi DHCP)
Display:	HDMI 1280x720
Stack:	LVGL 9.6 + fbdev + GCC 14.2
Build:	CMake 3.31 + Make 4.4
Created:	2026-05-06

เอกสารนี้สรุป workflow สำหรับการพัฒนา Digital Calendar บน Raspberry Pi
ครอบคลุมตั้งแต่ initial setup จนถึง daily development cycle

สารบัญ

ส่วนที่ 1	Daily Workflow (ใช้ทุกวันี่ทำงาน)	3
	1.1 ตอนเช้า: เริ่มต้นวันทำงาน	3
	1.2 ระหว่างวัน: Edit-Build-Run cycle	4
	1.3 ตอนเย็น: ปิดงาน	5
	1.4 Quick command reference	5
ส่วนที่ 2	Setup Workflow (ทำครั้งเดียว)	6
	2.1 Hardware preparation	6
	2.2 Pi OS installation	7
	2.3 SSH key authentication	8
	2.4 VS Code + SFTP setup	9
	2.5 Project initialization	10
	2.6 LVGL build first time	11
ส่วนที่ 3	Troubleshooting Reference	12
	3.1 Pi boot loop / undervoltage	12
	3.2 Network / SSH issues	12
	3.3 Build errors	13
ส่วนที่ 4	Concepts ที่ควารู้	14
	4.1 Software Stack คืออะไร	14
	4.2 CMake vs Make vs Compiler	14
	4.3 Source code vs Binary	15

ส่วนที่ 1: Daily Workflow

Workflow ที่ใช้ทุกวันทีกลับมทำงานบน project — ทำซ้ำได้เรื่อย ๆ

1.1 ตอนเช้า: เริ่มต้นวันทำงาน

ลำดับขั้นตอน:

1. เสียบ adapter ของ Pi → Pi เปิดอัตโนมัติ → รอ ~30 วินาที
2. Verify ว่า Pi connect WiFi LEnet ได้ (ดู router admin หรือ ping)
3. เปิด VS Code → File → Open Folder → D:\MY WORK\RASPBERRY PI PROJECT
4. เปิด PowerShell terminal (ใน VS Code: Ctrl+`)
5. ทดสอบ SSH:

```
ssh thanat@192.168.0.232 "uptime && vcgencmd get_throttled"
```

คาดหวัง: เห็น uptime + throttled=0x0 = Pi healthy พร้อมใช้งาน

1.2 ระหว่างวัน: Edit-Build-Run Cycle

ทุก 1 iteration ของ design GUI ใช้ลำดับนี้:

Step 1: Edit

เปิดไฟล์ใน VS Code (เช่น src/main.cpp) — แก้ตามต้องการ

Step 2: Save → Auto-upload

กด Ctrl+S → SFTP extension จะ upload ไฟล์ไปที่ Pi อัตโนมัติ (ดู notification ที่ status bar ล่าง)

Step 3: Build บน Pi

ที่ PowerShell terminal — รัน build:

```
ssh thanat@192.168.0.232 "cd ~/digital_calendar_pi/build && make -j1"
```

คาดหวัง: Incremental build ใช้เวลา 10-30 วินาที (recompile เฉพาะที่เปลี่ยน)

Step 4: Run

```
ssh thanat@192.168.0.232 "cd ~/digital_calendar_pi/build && ./bin/lvglsim"
```

ดูที่จอ HDMI ของ Pi — UI จะแสดงทันที

Step 5: Stop

กด Ctrl+C ที่ PowerShell — ส่ง SIGINT ไป Pi → app หยุด

Step 6: Repeat

กลับไป Step 1 — ทำซ้ำจนได้ผลที่ต้องการ (Senior dev ทำ cycle นี้ 50-100 ครั้งต่อวันเป็นเรื่องปกติ)

1.3 ตอนเย็น: ปิดงาน

เมื่อจะเลิกใช้งาน (ปิด Desktop หรือเลิกงาน) — shutdown Pi ปลอดภัยก่อน:

```
ssh thanat@192.168.0.232 "sudo shutdown -h now"
```

รอ ~30 วินาที — LED ของ Pi ดับสนิท → ถอด adapter ได้

เหตุผล: ถ้า cut power ขณะ Pi เปิด → SD card filesystem อาจ corrupt

1.4 Quick Command Reference

งาน	คำสั่ง
SSH เข้า Pi	ssh thanat@192.168.0.232
ตรวจสอบสุขภาพ Pi	ssh thanat@... "vcgencmd get_throttled; uptime"
Build (incremental)	cd ~/digital_calendar_pi/build && make -j1
Build (clean)	cd build && rm -rf * && cmake .. && make
Run app	cd ~/digital_calendar_pi/build && ./bin/lvglsim
Shutdown Pi	sudo shutdown -h now
ดู build log	tail -20 ~/digital_calendar_pi/build/build.log
ดู process รัน	ps -ef grep make
kill app	pkill -f lvglsim
tmux: เริ่ม session	tmux new -s build
tmux: detach	Ctrl+B [space] D
tmux: reattach	tmux attach -t build
tmux: list sessions	tmux ls

ส่วนที่ 2: Setup Workflow

Workflow ที่ทำครั้งเดียวตอนเริ่ม project — สำหรับ recover/reproduce บนเครื่องอื่น

2.1 Hardware Preparation

อุปกรณ์ที่ต้องมี:

- Raspberry Pi Zero WH (มี header soldered) หรือ Pi 4/5
- MicroSD card 16GB+ (Class 10 หรือสูงกว่า — ความเร็วสำคัญ)
- Power adapter 1.2A ขึ้นไป, แนะนำ 2A+ (สำคัญ!)
- Mini-HDMI to HDMI adapter (สำหรับ Pi Zero W)
- จอ HDMI
- USB keyboard (สำหรับ initial setup)
- MicroSD card reader (สำหรับ flash OS)
- Desktop/Laptop ที่มี WiFi เดียวกับ Pi

หมายเหตุสำคัญ: Pi Zero W ที่ใช้ adapter 800mA จะเจอ undervoltage → boot loop. ใช้ adapter ที่จ่ายไฟพอเสมอ
ตรวจด้วย `vcgencmd get_throttled`

2.2 Pi OS Installation

ขั้นตอน:

- Download Raspberry Pi Imager จาก [raspberrypi.com/software](https://www.raspberrypi.com/software)
- เสียบ SD ในการ์ดรีดเดอร์ → Desktop
- เปิด Raspberry Pi Imager → เลือก:
 - Device: Raspberry Pi Zero W/WH
 - OS: Raspberry Pi OS Lite (32-bit)
 - Storage: SD card
- คลิก Settings (gear icon) → ตั้ง:
 - Hostname: digitalcal-pi
 - Enable SSH: ✓ ใช้ password
 - Username/Password: ตั้งของตัวเอง
 - WiFi SSID/Password: ของบ้าน/ที่ทำงาน
 - Locale: Asia/Bangkok
- Write → รอ ~5-10 นาที
- ถอด SD → ใส่ Pi → เสียบ adapter → boot

2.3 SSH Key Authentication

เมื่อใช้ Pi เป็น dev server — SSH key auth ดีกว่า password (ไม่ต้องพิมพ์ทุกครั้ง + ปลอดภัยกว่า)

Step 1: ตรวจ SSH key เดิมก่อน (หรือสร้างใหม่)

```
dir $env:USERPROFILE\.ssh\
```

ถ้าเห็น id_ed25519 + id_ed25519.pub → ใช้ของเดิม ไม่ต้องสร้างใหม่ (1 key ใช้ได้กับหลาย server)

Step 2: ถ้ายังไม่มี — สร้าง

```
ssh-keygen -t ed25519 -C "thanat@desktop1"
```

กด Enter ทั้ง 3 ครั้ง (default path, no passphrase)

Step 3: Copy public key ไป Pi

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh thanat@192.168.0.232 "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

ใส่ password Pi ครั้งสุดท้าย — หลังจากนั้นไม่ถามอีก

Step 4: Verify

```
ssh thanat@192.168.0.232 "echo Hello from Pi"
```

ถ้าไม่ถาม password = ✓ สำเร็จ

2.4 VS Code + SFTP Setup

ทำไม SFTP ไม่ใช่ Remote SSH: Pi Zero W ใช้ ARMv6 architecture ที่ VS Code Remote SSH ไม่รองรับ. SFTP เป็น workaround ที่ทำงานได้ดี.

Step 1: Install SFTP extension

VS Code → Extensions → ค้นหา 'SFTP' by Natizyskunk → Install

Step 2: สร้าง project folder

สร้าง folder Windows สำหรับ project (เช่น D:\MY WORK\RASPBERRY PI PROJECT\) → เปิดใน VS Code

Step 3: สร้าง config

F1 → SFTP: Config → Enter — VS Code จะสร้างไฟล์ .vscode/sftp.json

แทนที่ด้วย:

```
{
  "name": "Pi Zero W - Digital Calendar",
  "host": "192.168.0.232",
  "protocol": "sftp",
  "port": 22,
  "username": "thanat",
  "privateKeyPath": "C:\\Users\\thana\\.ssh\\id_ed25519",
  "remotePath": "/home/thanat/digital_calendar_pi",
  "uploadOnSave": true,
  "useTempFile": false,
  "openSsh": false,
  "ignore": [".vscode", ".git", "build", "lvgl"]
}
```

Step 4: Sync ไฟล์จาก Pi → Windows ครั้งแรก

F1 → SFTP: Sync Remote → Local → เลือก profile

หลังจากนี้ — แก้ไฟล์ → Ctrl+S → SFTP upload ขึ้น Pi อัตโนมัติ

2.5 Project Initialization

ใช้ lv_port_linux template ของ LVGL official — มี config ที่ทดสอบมาแล้วสำหรับ Linux fbdev

Step 1: Clone template

```
ssh thanat@192.168.0.232 "cd ~ && git clone --recurse-submodules \\  
https://github.com/lvgl/lv_port_linux.git digital_calendar_pi"
```

คาดเวลา: ~3-5 นาที (clone LVGL submodule 62MB ผ่าน WiFi Pi Zero W)

Step 2: ตรวจ structure

```
ssh thanat@192.168.0.232 "ls ~/digital_calendar_pi/"
```

ควรเห็น: CMakeLists.txt, configs/, lvgl/, src/, lv_conf.defaults

Step 3: ใช้ fbdev config

```
ssh thanat@192.168.0.232 "cd ~/digital_calendar_pi && \\  
cp configs/fbdev.defaults lv_conf.defaults"
```

2.6 LVGL Build First Time

Step 1: cmake configure

```
ssh thanat@192.168.0.232 "cd ~/digital_calendar_pi && \\  
mkdir -p build && cd build && cmake .."
```

คาดเวลา: ~10-30 วินาที. ควรเห็น Configuring done, Generating done

Step 2: Build (ใน tmux เพื่อ detach ได้)

```
ssh thanat@192.168.0.232 -t "tmux new -s build \\  
'cd ~/digital_calendar_pi/build && make -j1; bash'"
```

คาดเวลา: 30-60 นาทีบน Pi Zero W (first build). Pi 5 ~5-10 นาที.

Detach: Ctrl+B แล้วกด D (build รันต่อ บน Pi)

Reattach: ssh thanat@... -t "tmux attach -t build"

Step 3: Run

หลังเห็น [100%] Linking C executable bin/lvglsim:

```
ssh thanat@192.168.0.232 "cd ~/digital_calendar_pi/build && ./bin/lvglsim"
```

ดูที่จอ HDMI — UI demo ของ LVGL ควรแสดง (Profile/Analytics/Shop tabs)

ส่วนที่ 3: Troubleshooting Reference

ปัญหาที่เคยเจอ + วิธีแก้ — สำหรับใช้กลับมาอ้างอิงเมื่อเจอปัญหาเดิม

3.1 Pi Boot Loop / Undervoltage

อาการ:

- Pi reboot ตัวเองซ้ำ ๆ
- ทำงานสักระยะแล้วดับ
- SD card filesystem corrupt หลัง install package

สาเหตุ:

Power adapter จ่ายไฟไม่พอ (เช่น 800mA) ทำให้ voltage drop ระหว่างมีโหลดเยอะ → SD write failure

Diagnose:

```
vcgencmd get_throttled
```

- 0x0 = ไม่มีปัญหา
- 0x50000 หรือเลขอื่น = เคยเจอ undervoltage

วิธีแก้:

- เปลี่ยน adapter เป็น 2A+ (ของแท้ Raspberry Pi Foundation ดีที่สุด)
- ถอด USB hub + peripherals ที่ไม่จำเป็น
- ใช้สาย USB คุณภาพดี (กระแสม่านได้เต็ม)

3.2 Network / SSH Issues

SSH: Connection refused

→ Pi อาจยังไม่ boot complete — รอ 30 วินาที

SSH: No route to host

→ Pi กับ Desktop อยู่คนละ subnet/WiFi. ตรวจสอบ:

- Desktop กับ Pi อยู่ WiFi เดียวกันไหม (ทั้งคู่ต้องเป็น LEnet)
- Network profile บน Windows = Private ไม่ใช่ Public
- Ping 192.168.0.232 ได้ไหม

เปลี่ยน Network Profile เป็น Private (Windows)

PowerShell as Admin:

```
Set-NetConnectionProfile -Name "LEnet" -NetworkCategory Private
```

3.3 Build Errors

Error: 'bad instruction' ใน lv_blend_helium.S

→ LVGL 9.x ใช้ ARM Helium SIMD ที่ Pi Zero W ARMv6 ไม่รองรับ

วิธีแก้: ใช้ lv_port_linux template (ไม่ใช่ raw lvgl/) — official template handle compatibility ให้แล้ว

Error: undefined reference to ...

→ Linker หา function ไม่เจอ. มักเป็น library ที่ลืม link.
ตรวจ `target_link_libraries(...)` ใน `CMakeLists.txt`

Build ค้างไม่จบ

→ Pi Zero W ซ้ำ. ใช้ `tmux + detach` แล้วเช็คทีหลัง:

```
pgrep -f make && echo "BUILDING" || echo "DONE"
```

ส่วนที่ 4: Concepts ที่ควรรู้

Concept พื้นฐานของ Software workflow — ใช้ได้กับทุก project ในอนาคต

4.1 Software Stack คืออะไร

Stack = ชั้นของ tools ที่ทำงานร่วมกัน เพื่อสร้างจาก source code → running program

Stack ของ project นี้:

Layer	Tool	หน้าที่
Application	LVGL	GUI library
Build config	CMakeLists.txt	Project description
Build generator	CMake	สร้าง Makefile
Build executor	make	เรียก compiler
Compiler	GCC	compile C/C++ → object
Linker	ld	รวม object → binary
Runtime	glibc	standard library
OS	Linux kernel	system calls
Hardware	ARM Pi	CPU + memory

4.2 CMake vs Make vs Compiler

ทั้ง 3 ตัวทำหน้าที่ต่างกัน:

CMake	Generator	อ่าน CMakeLists.txt → สร้าง Makefile (ทำงานครั้งเดียว)
Make	Executor	อ่าน Makefile → เรียก compiler ตามคิว (ทำงานทุกครั้งที่ build)
GCC	Compiler	แปลง .c/.cpp → .o (machine code) (make เรียกหลายครั้ง)

Workflow: CMake → Makefile → make → GCC → object → linker → binary

4.3 Source Code vs Binary

	Source Code	Binary
คืออะไร	ตัวอักษรที่อ่านได้	machine code 0/1
Portable	✓ ใช้ทุก platform	✗ ใช้แค่ platform ที่ build
ขนาด	เล็ก (~MB)	แล้วแต่ project
แก้ไข	edit ใน editor	ไม่ควรแก้

Backup	✓ สำคัญ — git push	△ rebuild ได้
Version control	✓ ใส่ git	✗ ไม่ใส่ git

Insight: เมื่อย้ายไป Pi 5 → binary ใช้ไม่ได้ ต้อง rebuild แต่ source code + CMakeLists ใช้ได้ทันที (rebuild ครั้งเดียว ~5-10 นาทีบน Pi 5)

สรุปสำคัญ

Workflow ที่ Lek ได้สร้างวันนี้ ประกอบด้วย:

1. Setup ทำครั้งเดียว:

- Hardware + OS
- SSH key + SFTP
- Project clone + first build

2. Daily cycle (ใช้ทุกวัน):

- Edit ใน VS Code → Save (auto-upload)
- SSH build (incremental ~10-30 วินาที)
- Run + verify ที่ HDMI
- Repeat

3. Tools ที่ติดตัวไปได้ตลอดชีวิต:

- PowerShell + SSH
- tmux
- VS Code + SFTP
- CMake + Make
- Git

4. Mindset ของ Software Engineer:

- "Iterate fast, fail fast"
- Source code + knowledge = asset, binary = disposable
- เลือก tool ตาม task ไม่ใช่ใช้ tool เดียวทำทุกอย่าง
- Logic ผิดที่ build = งานไม่สำเร็จ → debug ที่ root cause

5. ทักษะที่ใช้ได้กับทุก Linux project:

ทั้ง workflow และ concept ที่เรียนวันนี้ — apply ได้กับ Pi 5, embedded Linux อื่น ๆ, server Linux, container, cloud — ทุกที่ใช้ Linux stack.